

The current PDU architecture is simple, but too tightly coupled.

Current behavior first. Then the recommended refactor.

```
221\n
```

```
2 = SetSwitch
```

```
2 = SW_3V3_1
```

```
1 = ON
```

One small UART command drives real power hardware.

That is useful for demos, but it needs stronger boundaries before students treat it as plug-and-play mission infrastructure.

Today, the PDU speaks ASCII-offset bytes over UART.

It is not a readable command sentence. It is a tiny numeric protocol encoded as characters.

BYTE 1

Command byte

ASCII character minus 48
becomes the packet type.

BYTE 2

Switch byte

ASCII character minus 48
selects the rail or device.

BYTE 3

State byte

ASCII character minus 48
becomes off or on.

END

Newline

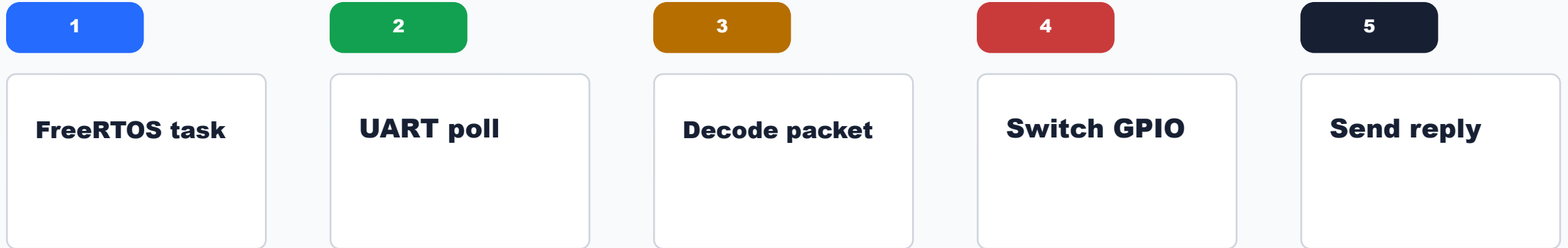
CR or LF ends the
command frame.

'2' - 48 = 2

'1' - 48 = 1

The active firmware path is one straight line.

This is why it is easy to demo and easy to outgrow.



The same handler currently mixes protocol parsing, command dispatch, physical GPIO behavior, and telemetry packing.

This is fine for learning the board, not for mission reuse.

The current design teaches the hardware path, but not a durable subsystem boundary.

CURRENT STRENGTH

Good today

- Fast to test
- Easy to inspect on serial
- Small enough for a demo
- Direct proof of GPIO behavior

CURRENT RISK

Hard tomorrow

- Raw struct assumptions
- No version or CRC
- Weak error model
- Physical rails leak upward

MENTAL LOAD

Student problem

They must reason about UART, enums, GPIO pins, composite rails, and telemetry layout at the same time.

Option 2 gives the PDU a real EPS interface.

The wire protocol should describe mission intent, not GPIO trivia.



```
magic | version | type | seq | len | payload | crc
```

The new commands should sound like EPS behavior.

Students should think in rails, health, faults, and safety actions.

PING

Prove the link is alive.

GET_CAPS

Report what this board supports.

SET_RAIL

Turn a named rail on or off.

GET_SOH

Return compact health state.

PULSE_RESET

Reset a supervised device.

GET_FAULTS

Report latched fault flags.

The PDU firmware becomes four small modules.

Each module has one job, so students know where to look.

MODULE

Transport

Read a frame.
Check CRC.
Return bytes.

MODULE

Protocol

Decode message type.
Validate payload.
Encode reply.

MODULE

EPS logic

Apply rules.
Reject unsafe state.
Build SOH.

MODULE

Board driver

Own pins.
Read sensors.
Hide hardware quirks.

No mission software should need to know that SW_12V also needs SW_5V_EN4.

The student reasoning path gets much cleaner.

Every layer answers a different question.

QUESTION 1

Mission layer

What do we want the spacecraft to do?

Example: turn payload power on.

QUESTION 2

Protocol layer

Was the request valid, versioned, acknowledged, and checked?

Example: SET_RAIL accepted.

QUESTION 3

Hardware layer

Which pins and sensors prove the rail actually changed?

Example: rail state and faults reported.

Recommendation: refactor to Option 2 first

Keep it small. Make the boundary real. Do not jump straight to a generator yet.

START

Step 1

Freeze the existing ASCII protocol as legacy v1.

Document examples and known limits.

BUILD

Step 2

Add a v2 framed EPS ICD.

Implement PING, GET_CAPS, SET_RAIL, GET_SOH.

INTEGRATE

Step 3

Write the Teensy client library.

Then connect F Prime through EpsService/EpsAdapter.

Best outcome: students reason through mission commands, protocol frames, and hardware proof separately.